

Pre-Hackathon Setup Guide

Please finish this before the event. It takes about 30-60 minutes. Setting up on hackathon day eats into your build time.

By the end, you'll have:

- A terminal you're comfortable opening
- git, a code editor, a GitHub account, and Node.js and/or Python
- At least one AI coding agent installed and logged in
- Drawa, a browser canvas for running and reviewing coding agents side by side
- A test project that runs

Stuck? Note the exact error message, take a screenshot, and bring it to the pre-event help session or post it in the event channel. "It doesn't work" is hard to fix. The error text usually isn't.

Step 0: Pick your path

macOS: use the built-in Terminal app. Skip to Step 1.

Linux: use your usual terminal. Skip to Step 1.

Windows: set up WSL2 first (below), then do everything in the Ubuntu terminal.

Why WSL instead of plain Windows?

WSL2 (Windows Subsystem for Linux) runs a real Ubuntu Linux inside Windows. It's free, built by Microsoft, and you keep using Windows normally. We ask Windows users to use it because:

- **Drawa doesn't run on native Windows yet.** It supports macOS and Linux, so WSL is how Windows users get it.
- **Most install commands and tutorials assume a Linux or macOS terminal.** Commands like `curl ... | bash`, `source ~/.bashrc`, and `sudo apt install` don't work in PowerShell. In WSL they work exactly as written in this guide.
- **AI agents write better commands for Linux.** Coding agents run shell commands for you, and they're most reliable in a bash environment. On Windows they more often trip over paths (`\` vs `/`), line endings, and missing Unix tools.
- **Your app will likely be deployed on Linux.** Hosting platforms like Vercel, Render, and Railway run Linux, so building in WSL means fewer "worked on my laptop, broke in production" surprises.
- **One setup for the whole team.** When Mac, Linux, and Windows teammates all run the same commands, helping each other is much easier.

You can still use Windows apps like VS Code, Chrome, and Edge. They connect to WSL seamlessly.

Windows only: install WSL2

1. Open **PowerShell as Administrator** (right-click Start → *Terminal (Admin)*).
2. Run:

```
wsl --install
```

3. **Restart** your computer.
4. Open **Ubuntu** from the Start menu. It will ask you to make a Linux username and password. *Remember the password.* You'll need it for sudo, and the screen stays blank while you type it. That's normal.
5. Update Ubuntu:

```
sudo apt update && sudo apt upgrade -y
```

From here on, **Windows users: run every command in the Ubuntu terminal**, not PowerShell. Keep your projects in your Linux home folder (~/projects), not under /mnt/c/, because that's much slower.

Step 1: Install the basics

Everyone installs: git (version control), the GitHub CLI, and a code editor.

Then Node.js, Python, or both, depending on what your team builds with.

Node.js or Python?

Both can build a complete full-stack app, so pick based on your team's language and project idea, not on frontend vs backend.

Install Node.js if your team will write JavaScript or TypeScript: React, Next.js, Vue, Svelte, Express, or a browser extension. A Next.js app alone covers frontend, backend, and API routes. You also need Node if you plan to use the **Codex** agent, which installs through npm.

Install Python if your team will write Python: FastAPI, Flask, or Django backends, Streamlit or Gradio for quick UIs, or anything with data, machine learning, or AI libraries (pandas, PyTorch, Hugging Face, LangChain). Streamlit and Gradio let you build a working web app without writing any JavaScript.

Install both if you're not sure yet, your team hasn't picked an idea, or you want a React frontend with a Python backend. That's a common hackathon combo. It only takes a few extra minutes, and AI agents sometimes reach for the other language for a helper script anyway.

Our recommendation: install both. You won't lose anything, and you won't get stuck mid-hackathon.

macOS

First install Homebrew, the macOS package manager. It will also install Apple's command line tools, which can take a few minutes.

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

When it finishes, it prints two or three "Next steps" commands to add brew to your PATH. **Run them**, then:

```
brew install git gh      # everyone
brew install node        # Node.js
brew install python      # Python
```

Linux / Windows (WSL Ubuntu)

git and build tools

```
sudo apt update
sudo apt install -y git curl build-essential
```

Node.js (if you need it) via nvm, which avoids sudo and permission errors with npm:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
source ~/.bashrc
nvm install --lts
```

Python (if you need it), plus pip and virtual environments:

```
sudo apt install -y python3 python3-pip python3-venv
```

GitHub CLI

```
sudo apt install -y gh
```

Code editor

Install [VS Code](#) (or any editor you already like).

- **Windows:** install VS Code on Windows itself, not inside Ubuntu, then add the **WSL** extension. Afterwards, typing `code .` in the Ubuntu terminal opens that folder in VS Code.
- **macOS:** open VS Code, press `Cmd+Shift+P`, and run **Shell Command: Install 'code' command in PATH** so `code .` works from the terminal.

Check it worked

```
git --version
node --version
npm --version
python3 --version
gh --version
code --version
```

Each one should print a version number. Skip node/npm or python3 if you didn't install them. You want Node 20 or newer and Python 3.10 or newer.

Python tip: use a virtual environment per project

On recent Ubuntu and macOS, `pip install` outside a virtual environment is blocked with an "externally managed environment" error. That's expected. Make a `venv` inside each project instead:

```
python3 -m venv .venv
source .venv/bin/activate
pip install requests          # now this works
```

Run `source .venv/bin/activate` again each time you open a new terminal for that project.

Tell git who you are

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Step 2: Create accounts

- ☐ **GitHub:** github.com/signup. Use the same email as above.
- ☐ Log in to GitHub from the terminal (optional, recommended):

```
gh auth login
```

- ☐ **An AI plan** for at least one agent (see Step 3). Check whether your hackathon provides credits before paying for anything.

Step 3: Install at least one coding agent

You only **need one**. Pick based on what you have access to.

IF YOU HAVE...	INSTALL
A Claude Pro/Max plan	Claude Code
A ChatGPT plan or OpenAI API key	Codex
No paid plan, or another provider (Google, OpenRouter, etc.)	OpenCode (has free models)

Claude Code

```
curl -fsSL https://claude.ai/install.sh | bash
claude --version
claude          # first run opens a browser to log in
```

Docs: code.claude.com/docs/en/setup

Codex

```
npm install -g @openai/codex
codex login
```

Docs: developers.openai.com/codex/cli

OpenCode

```
curl -fsSL https://opencode.ai/install | bash
opencode auth login      # skip if you only want the free models
```

Docs: opencode.ai/docs

Privacy note on free models: OpenCode's free models are run by third parties that may use what you send to improve their models. Don't paste API keys, passwords, or code you can't share.

Windows reminder: install your agent *inside Ubuntu*. A copy installed in Windows/PowerShell won't be visible to Drawa.

Step 4: Install Drawa

```
curl -fsSL https://raw.githubusercontent.com/HimalayanNomads/drawa/main/install.sh | sh
```

If drawa isn't found afterward, add it to your PATH:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc && source ~/.bashrc
```

(macOS users on the default zsh shell: replace `~/.bashrc` with `~/.zshrc`.)

Step 5: The smoke test

This is the important part. If this works, you're ready.

```
mkdir -p ~/projects/hello-hackathon
cd ~/projects/hello-hackathon
git init
drawa
```

1. Your browser should open `http://127.0.0.1:8765`. On Windows, open that address in Chrome or Edge yourself if it doesn't open automatically.
2. Press **N** to start a new session.
3. Type:

Create a simple `index.html` page that says "Ready for the hackathon!" with a nice gradient background.

4. Approve the edit when the agent asks.
5. Open `index.html` in your browser and confirm you see the page.

Done! Take a screenshot of the Drawa canvas and post it in the event channel so organizers know you're set.

Troubleshooting

PROBLEM	TRY THIS
command not found after installing something	Close and reopen the terminal, or run <code>source ~/.bashrc</code> (<code>~/.zshrc</code> on macOS).
permission denied with npm	Don't use <code>sudo npm</code> . On Linux/WSL, use <code>nvm</code> as shown in Step 1.
externally-managed-environment from pip	Use a virtual environment (see the Python tip in Step 1).
brew: command not found right after installing Homebrew	Run the "Next steps" commands the Homebrew installer printed, then reopen the terminal.
code: command not found	macOS: run the VS Code PATH command from Step 1. Windows: install the WSL extension and reopen Ubuntu.
Agent says you're not logged in	Run its login command again (<code>claude</code> , <code>codex login</code> , or <code>opencode auth login</code>).
Drawa page won't load on Windows	Make sure drawa is still running in the Ubuntu terminal, then visit <code>http://127.0.0.1:8765</code> manually.
WSL install fails	Make sure virtualization is enabled in your BIOS and Windows is up to date.
Something else	Copy the full error, screenshot it, and ask in the event channel.

Before you leave home

- ☐ Smoke test passed
- ☐ Laptop **and** charger
- ☐ Logged in to GitHub and your AI agent (check it still works)
- ☐ Phone hotspot as a Wi-Fi backup

See you there!